Regularization of a Simple Neural Network

Basic feedforward networks

Mohib Ullah

12 March, 2026

Outline

1. Setting and motivation
2. General idea of regularization
3. L2 regularization
4. L1 regularization
5. Early stopping
6. Dropout
7. Other simple regularizers
8. Choosing hyperparameters
9. Worked mathematical views

What neural network are we studying?

We will focus on a **simple feedforward neural network**, for example a one-hidden-layer model:

$$\hat{y} = f(x;\theta) = W_2\,\sigma(W_1 x + b_1) + b_2$$

where:

- $x \in \mathbb{R}^d$ is the input,
- $\sigma(\cdot)$ is an activation function,
- $\theta = \{W_1, b_1, W_2, b_2\}$ are the trainable parameters.

We stay at this level instead of moving into large deep learning systems.

Supervised learning setup

Suppose we have training data

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}.$$

Our goal is to find parameters θ such that the network prediction $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ is close to the true target $y^{(i)}$.

- Regression: often use mean squared error.
- Classification: often use cross-entropy.

Main question

How do we train a network that fits the training data *and* still works well on unseen data?

Forward propagation in a simple network

For one hidden layer:

$$z_1 = W_1 x + b_1, \qquad h = \sigma(z_1), \qquad \hat{y} = W_2 h + b_2.$$

- The hidden vector h is an internal representation.
- The output $\hat{y}$ depends on both layers of weights.
- If the network has too much freedom, it may memorize the training set.

Loss without regularization

For regression with mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - f(x^{(i)}; \theta) \right)^2 .$$

For binary classification with cross-entropy:

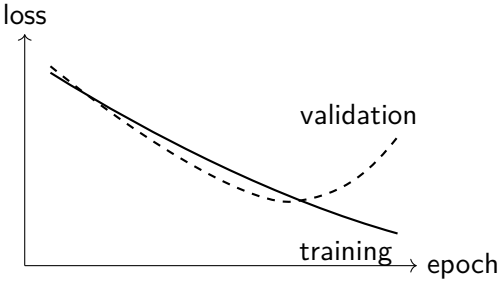
$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] .$$

Problem

Minimizing the training loss alone may produce a network that is excellent on training data but poor on new examples.

What does overfitting look like?

- Training loss keeps decreasing.
- Validation loss decreases at first, then starts increasing.
- The network starts fitting noise, accidental patterns, or outliers.



Bias-variance viewpoint

- A very rigid model has high bias.
- A very flexible model has high variance.
- Regularization usually adds some bias to reduce variance.

$$\text{generalization error} \approx \text{bias}^2 + \text{variance} + \text{noise}$$

Interpretation

A slightly more constrained network can generalize better than a very flexible one.

Regularized objective

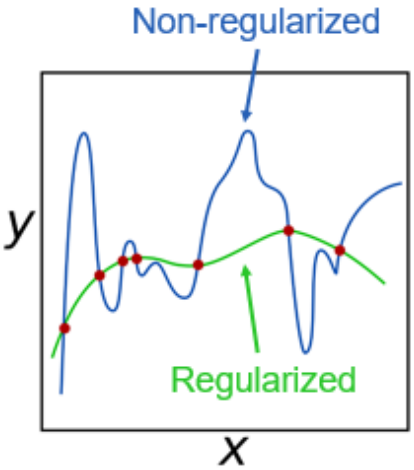
Instead of minimizing only the data loss, we minimize

$$\mathcal{J}(\theta) = \mathcal{L}(\theta) + \lambda\,\Omega(\theta)$$

where:

- $\mathcal{L}(\theta)$ measures fit to the training data,
- $\Omega(\theta)$ penalizes complexity,
- $\lambda \geq 0$ controls the strength of the penalty.

Interpretation: fit the data well, but not with an unnecessarily complicated parameter choice.



What does the penalty term do?

- It changes the optimization target.
- It discourages certain kinds of parameter values.
- It can make the learned function smoother, simpler, or more stable.

For example, if large weights are penalized, the network is less likely to create extremely sharp changes in output.

L2 regularization or weight decay

For a simple neural network, L2 regularization is

$$\mathcal{J}(\theta) = \mathcal{L}(\theta) + \lambda \|\theta\|_2^2$$

with

$$\|\theta\|_2^2 = \sum_k \theta_k^2.$$

- Large weights are penalized more strongly.
- The optimizer prefers smaller parameter values.
- This often improves generalization in practice.

L2 for a one-hidden-layer network

If

$$\theta = \{W_1, b_1, W_2, b_2\},$$

then one common choice is

$$\Omega(\theta) = \|W_1\|_F^2 + \|W_2\|_F^2,$$

so that

$$\mathcal{J}(\theta) = \mathcal{L}(\theta) + \lambda \left(\|W_1\|_F^2 + \|W_2\|_F^2 \right).$$

- $\|\cdot\|_F$ is the Frobenius norm.
- Biases are sometimes penalized, but often only the weights are regularized.

Gradient effect of L2

Suppose the original gradient is

$$\nabla_\theta \mathcal{L}(\theta).$$

With L2 regularization,

$$\nabla_\theta \mathcal{J}(\theta) = \nabla_\theta \mathcal{L}(\theta) + 2\lambda\theta.$$

A gradient descent update becomes

$$\theta \leftarrow \theta - \eta \left(\nabla_\theta \mathcal{L}(\theta) + 2\lambda\theta\right)$$

which can be written as

$$\theta \leftarrow (1 - 2\eta\lambda)\theta - \eta \nabla_\theta \mathcal{L}(\theta).$$

Meaning: each step slightly shrinks the weights.

Why shrinking weights can help

- Very large weights can create unstable responses to small input changes.
- Smaller weights often imply smoother mappings $x \mapsto \hat{y}$.
- The model is less likely to create highly specific rules for isolated training points.

Intuition

Regularization does not stop learning. It simply prefers simpler parameter settings among all settings that fit the data.

A tiny scalar example for L2

Suppose the loss for one parameter w is

$$\mathcal{L}(w) = (w-5)^2.$$

Without regularization, the minimum is at $w = 5$.
With L2 regularization:

$$\mathcal{J}(w) = (w-5)^2 + \lambda w^2.$$

Set derivative to zero:

$$\frac{d\mathcal{J}}{dw} = 2(w-5) + 2\lambda w = 0.$$

Therefore,

$$(1+\lambda)w = 5 \qquad \Rightarrow \qquad w^* = \frac{5}{1+\lambda}.$$

So the optimal weight is pulled toward zero.

L1 regularization

L1 regularization uses

$$\mathcal{J}(\theta) = \mathcal{L}(\theta) + \lambda \|\theta\|_1, \qquad \|\theta\|_1 = \sum_k |\theta_k|.$$

- Encourages sparsity.
- Some weights may become exactly zero.
- This can simplify the network or highlight important connections.

L1 vs L2 in simple words

Method	Main effect	Typical interpretation
L2	Shrinks all weights smoothly	Prefer smaller, smoother solutions
L1	Pushes many weights toward zero	Prefer sparse solutions

- L2 is usually the first regularizer to try.
- L1 is useful when sparsity is desirable.

Elastic net idea

A mixture of L1 and L2 is called elastic net:

$$\mathcal{J}(\theta) = \mathcal{L}(\theta) + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2.$$

- L1 gives sparsity.
- L2 gives smooth shrinkage.
- The combination can be useful when we want both effects.

Early stopping as regularization

- Train the network while monitoring validation loss.
- Stop before the network begins to overfit.
- Keep the parameter values from the best validation epoch.

Idea

The network may first learn the signal and later start fitting noise. Early stopping prevents that late phase from dominating.

Why early stopping is mathematically reasonable

Suppose we run gradient descent:

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t).$$

If we train for too many steps, the optimizer may continue reducing training error by fitting noise.

Stopping at a suitable time acts like a constraint on how far the parameters are allowed to move from initialization.

Practical early-stopping procedure

1. Split data into training and validation sets.
2. Train the network for many epochs.
3. Measure validation loss after each epoch.
4. Save the best-performing checkpoint.
5. Stop after a chosen patience value.

Dropout in a simple hidden layer

Let the hidden activation vector be h. During training, dropout replaces it by

$$\tilde{h} = m \odot h, \qquad m_j \sim \text{Bernoulli}(1-p).$$

Here:

- p is the dropout rate,
- $m_j = 0$ means neuron j is turned off,
- $m_j = 1$ means neuron j stays active.

Expected value with inverted dropout

A common version uses

$$\tilde{h} = \frac{m \odot h}{1-p}.$$

Then for each component,

$$\mathbb{E}[\tilde{h}_j] = \mathbb{E}\left[\frac{m_j h_j}{1-p}\right] = \frac{h_j}{1-p}\mathbb{E}[m_j] = \frac{h_j}{1-p}(1-p) = h_j.$$

Why this matters

The expected activation at training time matches the activation used at test time.

Why dropout can reduce overfitting

- Hidden units cannot rely too strongly on specific other units.
- The network learns more robust internal representations.
- It behaves like training many related subnetworks and sharing parameters among them.

Important

Dropout is applied during training, not during final inference.

Where dropout is useful in a simple ANN

- Often useful in hidden fully connected layers.
- Less useful if the network is already very small.
- Too much dropout can make optimization unstable or slow.

Rule of thumb

Try mild values first, such as $p = 0.1$ to 0.5, and use validation performance to decide.

Noise injection

Add small random noise to the input:

$$\tilde{x} = x + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

Then train the network on $\tilde{x}$ instead of x.

- Encourages local smoothness.
- Makes the network less sensitive to tiny perturbations.
- Very natural for a simple feedforward network.

Data augmentation in a simple setting

Even for a basic neural network, regularization can come from changing the data:

- add small noise,
- perturb numeric features slightly,
- rebalance examples,
- create simple transformed inputs when the label should not change.

Effect: the network learns the concept, not just the exact training examples.

Label smoothing for classification

If the target class is k among K classes, instead of a hard one-hot target we use:

$$y_i = \begin{cases} 1 - \varepsilon & i = k, \\ \frac{\varepsilon}{K-1} & i \neq k. \end{cases}$$

- Reduces overconfidence.
- Helps when the classifier becomes too certain too early.
- ε could be 0.05.

How do we choose λ?

- If $\lambda = 0$, there is no penalty.
- If λ is too large, the network may underfit badly.
- The best value is found from validation experiments.

Value of λ	Typical effect	Risk
very small	weak regularization	overfitting
moderate	balanced behavior	often useful
too large	strong shrinkage	underfitting

How do we choose the dropout rate p?

- $p = 0$ means no dropout.
- Larger p means more hidden units are removed during training.
- If p is too high, the model may learn too slowly or fail to fit the data.

$$\text{active fraction} = 1 - p$$

So with $p = 0.4$, only 60% of units are active on average during each training step.

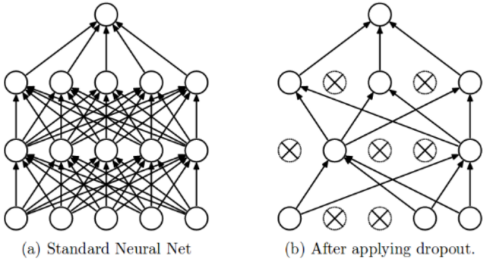


Figure 1: Dropout applied to a Standard Neural Network (Image by Nitish)

Underfitting vs overfitting vs good fit

- **Underfitting:** both training and validation performance are poor.
- **Overfitting:** training is strong, validation is weak.
- **Good fit:** both training and validation are good, with a small gap.

Practical point

Regularization should reduce the generalization gap without making the network too weak to learn the true pattern.

Worked example: one-neuron network

Consider a single neuron:

$$\hat{y} = wx + b.$$

For one data point (x, y) with squared error and L2 regularization:

$$\mathcal{J}(w, b) = (y - (wx + b))^2 + \lambda w^2.$$

Then

$$\frac{\partial \mathcal{J}}{\partial w} = -2x(y - (wx + b)) + 2\lambda w, \qquad \frac{\partial \mathcal{J}}{\partial b} = -2(y - (wx + b)).$$

Observation: the penalty changes the weight update, but not the bias update in this example.

Worked example: one-hidden-layer network with L2

Let

$$\hat{y} = W_2\sigma(W_1x + b_1) + b_2.$$

Use the objective

$$\mathcal{J}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 + \lambda\left(\|W_1\|_F^2 + \|W_2\|_F^2\right).$$

During backpropagation, the data-loss gradients are modified as

$$\frac{\partial\mathcal{J}}{\partial W_1} = \frac{\partial\mathcal{L}}{\partial W_1} + 2\lambda W_1, \qquad \frac{\partial\mathcal{J}}{\partial W_2} = \frac{\partial\mathcal{L}}{\partial W_2} + 2\lambda W_2.$$

Why the math matches the intuition

- The data term says: fit the samples.
- The regularization term says: do not use unnecessarily large or unstable parameters.
- The optimizer balances these two pressures at every update step.

$$\text{fit pressure} + \text{simplicity pressure} \longrightarrow \text{better generalization}$$

A practical recipe for a simple ANN

1. Start with a small or moderate one-hidden-layer network.
2. Split the data into training and validation sets.
3. Train with a baseline optimizer.
4. Add L2 regularization first.
5. If needed, add early stopping and mild dropout.
6. Tune λ and p using validation performance.

Common mistakes

- Using too much regularization and causing underfitting.
- Choosing hyperparameters using the test set.
- Forgetting to turn off dropout during evaluation.
- Using no validation set.
- Believing higher training accuracy always means a better model.

A compact comparison table

Method	Main mathematical form	Main effect
L2	$\mathcal{L} + \lambda \|\theta\|_2^2$	smooth shrinkage of weights
L1	$\mathcal{L} + \lambda \|\theta\|_1$	sparsity
Early stopping	stop at best validation epoch	implicit control of complexity
Dropout	$\tilde{h} = m \odot h$	robustness of hidden units
Noise injection	$\tilde{x} = x + \varepsilon$	local smoothness

Conceptual summary

- Regularization is not a single trick.
- It is a general strategy for controlling model complexity.
- In a simple neural network, the most common tools are:
  - L2 weight decay,
  - L1 sparsity,
  - early stopping,
  - dropout,
  - noise or data perturbation.

Key mathematical takeaway

Without regularization:

$$\min_{\theta} \mathcal{L}(\theta)$$

With regularization:

$$\min_{\theta} \left[\mathcal{L}(\theta) + \lambda\Omega(\theta)\right]$$

The extra term $\lambda\Omega(\theta)$ is what turns simple training into controlled training.

Final takeaway

- A simple neural network can still overfit.
- Regularization helps by limiting harmful complexity.
- The math explains the mechanism clearly.
- The best choice is guided by validation performance.

Thank you